

O'REILLY®

Building Integrated AI Agents with OpenClaw

Sajal Sharma



About Me

- AI Engineer with 8+ years in ML, NLP & Software Engineering - across product and consulting
- Currently building AI-agents for various ventures at Liminal (prev. Menyala) - a venture studio in Singapore
- Teaching:
 - [Building AI Agents with LangGraph](#)
 - [AI Agent Verifiable Skill](#)
 - [Agentic RAG with LangGraph](#),
 - [Intro to Claude Agent SDK](#)
- Mentor at Udacity and University of Melbourne | Guest Lecturer at Yale
- Passionate about crafting composable agentic systems & sharing knowledge through sajalsharma.com



[LinkedIn](#) | Email: contact@sajalsharma.com



Learning Outcomes

- Understand OpenClaw's architecture and design philosophy
- Be able to deploy, customize, and extend a multichannel AI agent
- Implement security guardrails for autonomous agents
- Build proactive automation and multi-agent workflows

Course schedule

- **Module 1: The Integrated AI Agent & OpenClaw Architecture**
 - Evolution to integrated agents, OpenClaw intro, architecture deep dive
 - **Demo:** OpenClaw in Action
- **Module 2: Deployment Options & Going Live**
 - Where does OpenClaw run? Deployment tradeoffs, security considerations
 - **Demo:** VPS Retrospective + Dashboard Deep Dive
- **Module 3: The OpenClaw Workspace: Tailoring Your Agent**
 - Workspace deep dive, bootstrap files, skills system, plugins/MCP/models
 - **Demo:** Context Files, Adding a Custom Skill
- **Module 4: Agent Security & Proactive Automation**
 - Agent security layer, guardrails, proactive automation, multi-agent routing
 - **Demo:** Security pitfalls and guardrails in action, Automation & Multi Agent Routing



Module 1: The Integrated AI Agent & OpenClaw Architecture



Introduction to OpenClaw

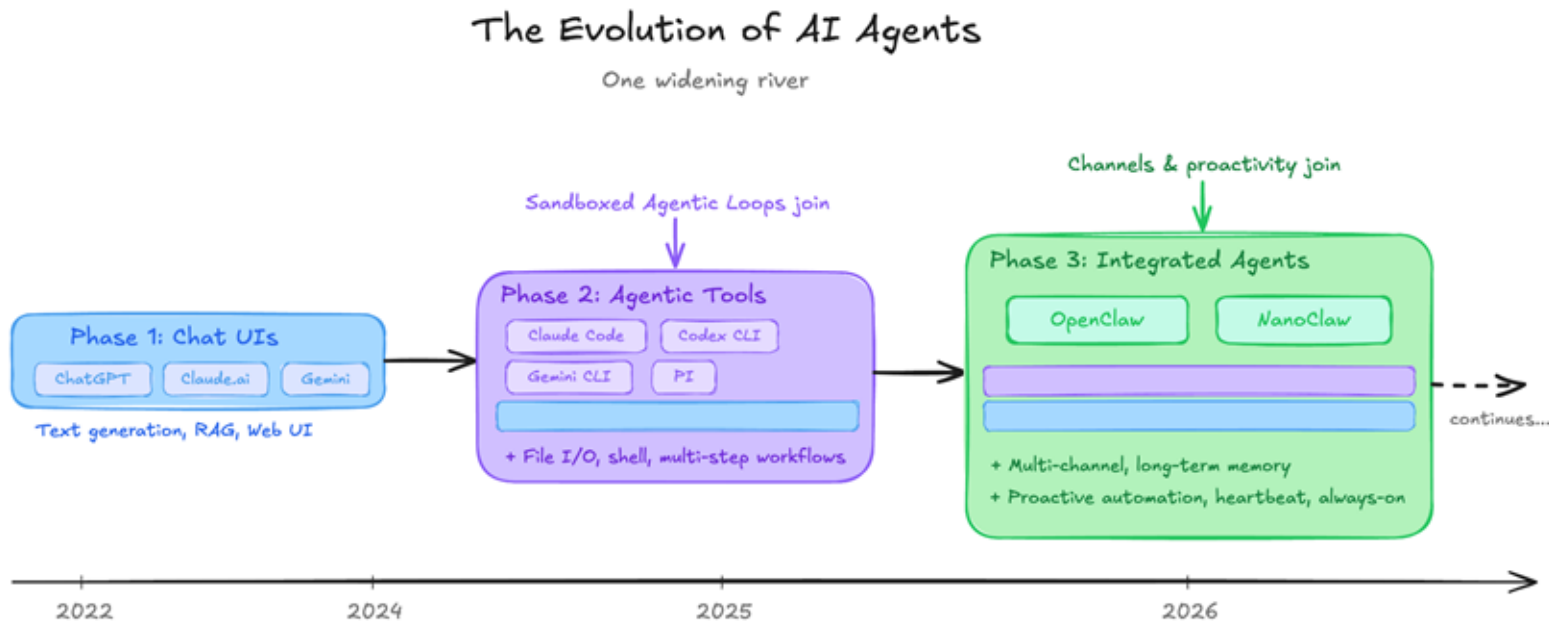
What is OpenClaw?

Officially, OpenClaw is a **self-hosted gateway** that connects messaging apps to AI agents. In practice, it is a **personal AI assistant** you run on your own devices.

- A single Gateway process bridges WhatsApp, Telegram, Slack etc to an embedded AI agent
- Runs on your hardware: laptop, home server, VPS
- All data stays on your infrastructure (only LLM API calls leave the host)
- Open-source, 340,000+ GitHub stars by April 2026
- Created by Peter Steinberger (Nov 2025, originally "Clawdbot")

Two framings: the gateway framing (architecture: routes messages between channels and agents) and the assistant framing (experience: an always-on AI in your chat window). The gateway makes the assistant possible.

The Evolution: Three Phases of AI Agents



Each phase widens the river — capabilities accumulate

Phase 1: Chat UIs

- **Conversational AI for everyone:** ChatGPT, [Claude.ai](#), Gemini made LLM interaction mainstream
- **Multimodal inputs:** Image upload, file attachments, vision capabilities (2023+)
- **Tool calling:** Function calling APIs (mid-2023+) gave models access to web search, code interpreters, third-party APIs
- **Self-hosted options:** Open WebUI (124K+ stars), LobeChat, [Jan.ai](#) replicated the experience locally

Chat UIs: Limitations

- **No open-ended environment:** Tools existed, but only as **platform-provided interfaces** (a search plugin, a code sandbox).
- **Models not built for sustained tool use:** Multi-turn tool calling became technically possible, but models were not optimized for long chains of reasoning + acting
- **Ecosystem lagged behind:** MCP (late 2024) and skill/plugin ecosystems that would make complex tool orchestration practical arrived at the tail end of this phase or after it
- **No persistent workspace:** No filesystem to build up artifacts over time, no ability to write code, run it, observe results, and iterate

Phase 2: Agentic Chat Tools

- **Sandboxed environments:** Agents gained terminals, filesystems, and shell access, no longer confined to platform-provided tool interfaces
- **Models optimized for tool use:** LLMs evolved to handle multi-tool calling, long-running conversations, and sustained chains of reasoning + acting + observing
- **The agentic loop:** LLM drives the loop, not the human. Reason -> act -> observe -> repeat. It can encounter errors, change strategy, and continue
- **Autonomous multi-step workflows:** Claude Code, Codex CLI, Gemini CLI read codebases, write files, run tests, and iterate inside terminals
- **General-purpose use:** Built for coding, but users quickly applied them to data analysis, file management, research

Agentic Chat Tools: Limitations

- **Interface-bound:** Only accessible through a terminal or desktop app
- **No messaging integration:** No way to reach the agent from WhatsApp, Telegram, or Slack
- **Session-dependent:** Agent only runs while the user has a session open.
No always-on presence
- **No proactive behavior:** Agent waits to be prompted; it cannot initiate actions on its own

Phase 3: Integrated Agents

- **Broke out of the terminal:** Same agentic loop, now reachable through **WhatsApp, Telegram, Slack, Discord**, and other messaging channels
- **Always-on availability:** Runs continuously as a service, not just when a user opens a session
- **Persistent memory:** Remembers across sessions and conversations via session transcripts and long-term memory files
- **Proactive automation:** Heartbeat checks, cron jobs, webhooks. The agent initiates actions without being prompted
- **Self-hosted and customizable:** Full data sovereignty, workspace-based personality and behavior configuration

OpenClaw's original pitch: "**connecting WhatsApp to Claude Code**", taking the agentic loop and making it accessible in the chat apps people already use

Integrated Agents: Limitations

- **Massively expanded attack surface:** A Phase 2 agent required local terminal access; a Phase 3 agent is reachable by **anyone who can send a message**.
- **The trust boundary problem:** The agent processes natural language from external sources (messages, webhooks, group chats) and translates it into real actions (shell commands, file writes). No reliable way to separate legitimate instructions from injected ones
- **Operational complexity:** Always-on means always needing monitoring: uptime, crashes, rate limits, resource usage, and LLM API costs for proactive tasks that run even when nobody is asking
- **Early and fast-moving:** This phase is months old; security models, best practices, and tooling are still being established

Why OpenClaw Exploded

- **Timing:** users wanted AI integrated into their daily messaging workflow
- **The WhatsApp connection:** 2B+ users; turning WhatsApp into an AI agent interface was a powerful hook
- **Open-source, self-hosted:** full data sovereignty in a market of proprietary cloud services
- **Low barrier to entry:** basic Telegram bot running in under 15 minutes
- **Community ecosystem:** ClawHub: 13,700+ skills by February 2026

The Alternatives Landscape

- **ZeroClaw**: Written in Rust, ships as a 3.4 MB binary running on <5 MB RAM. Targets extreme resource efficiency, but has a smaller plugin/skills ecosystem
- **NanoClaw**: Docker-isolated, ~4K lines of code, built on Claude Code SDK. Born specifically to address OpenClaw's security concerns, but less feature-rich
- **Moltworker**: Cloudflare's adaptation of OpenClaw for Cloudflare Workers. Serverless deployment on the edge, but a hybrid cloud model rather than fully self-hosted

Adjacent categories (overlapping but different)

- **Chat UIs** (Open WebUI, LobeChat): no channels, no proactive automation
- **Agent frameworks** (LangChain, CrewAI, AutoGen): libraries requiring custom assembly
- **Coding agents** (Claude Code, Cursor): terminal-focused, no integration for messaging channels
- **Workflow automation** (n8n, Zapier): workflow-triggered, require predefined pipelines



Poll

How are you currently using AI agents day-to-day?

- Chat-only (ChatGPT, Claude.ai, etc.)
- Coding agents (Claude Code, Cursor, Copilot)
- Self-hosted / open-source agents
- Built custom agents with SDKs (LangGraph, Agent SDK, etc.)
- Not yet, here to learn

Demo: OpenClaw in Action



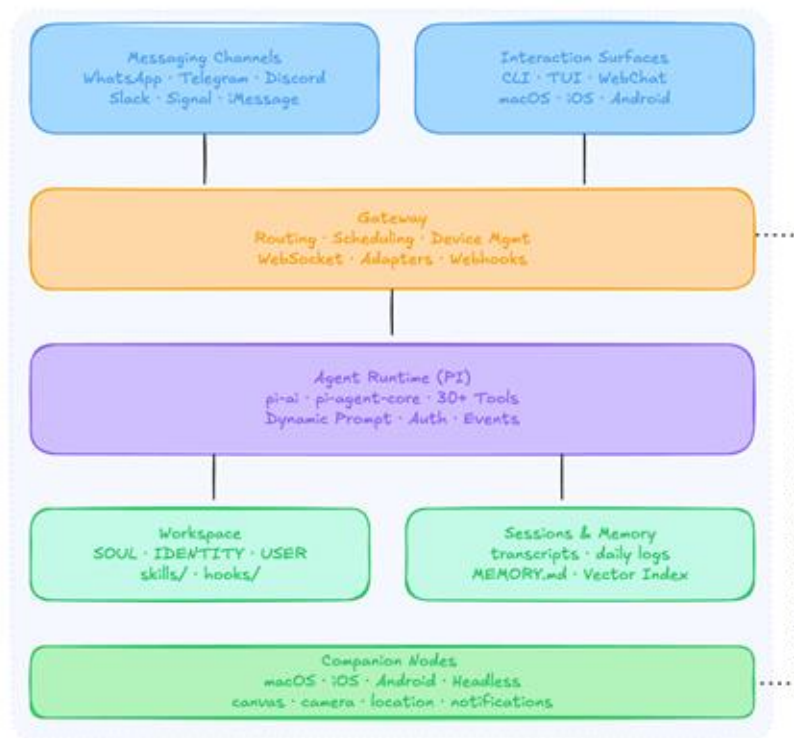


OpenClaw Architecture

Architecture



OpenClaw Architecture The Big Picture



The Gateway

The Gateway is the single point of coordination. Channels, agents, nodes, and clients all connect to it.

- A **long-lived Node.js daemon** that owns everything
- **Channel connections**: maintains persistent connections to all messaging platforms
- **Message routing**: resolves inbound messages to session keys, dispatches to agent runtime
- **Scheduling**: runs the cron scheduler and heartbeat system
- **Device management**: companion node pairing, capability discovery, command routing
- **Config hot-reload**: applies config changes live without full restart

Agent Runtime: Pi

OpenClaw does not implement its own agent reasoning. It embeds PI (pi-mono) as its core engine. PI handles LLM calls, tool execution, and the agentic loop; OpenClaw handles everything else.

- **pi-ai**: Unified multi-provider LLM streaming (Anthropic, OpenAI, Google, Mistral, Ollama, 2000+ models)
- **pi-agent-core**: The agent loop (LLM call -> tool execution -> feed results -> repeat)
- **pi-coding-agent**: Full SDK: session persistence, auth, model registry, built-in tools
- **pi-tui**: Terminal UI framework

PI was created by Mario Zechner. Github Repo: <https://github.com/badlogic/pi-mono>

Workspace

The workspace is a directory on disk that defines who the agent is, what it knows, and how it behaves, entirely through Markdown files and folder conventions.

- **Bootstrap / Context files** injected into system prompt each run: **SOUL.md**, **IDENTITY.md**, **USER.md**, **AGENTS.md**, **TOOLS.md**, **HEARTBEAT.md**
- Plus **skills/**, **hooks/**, **memory/** directories
- File-based configuration: edit Markdown files to change agent personality, knowledge, and behavior

Don't worry about the details for now. We will cover all this in Module 3.

Sessions and Memory

Sessions track individual conversations; memory gives the agent durable knowledge that persists across sessions and restarts.

Sessions (conversation state)

- Each session = a distinct conversation thread (DM, group, cron job, sub-agent)
- **Session store** (`sessions.json`): lightweight metadata index
- **Transcripts** (`<sessionId>.jsonl`): append-only with tree structure for branching

Memory (durable knowledge)

- **Daily logs**, `MEMORY.md`
- **Vector index** allowing agent to recall relevant facts

Interaction Surfaces

Besides messaging channels, OpenClaw provides multiple ways to interact with the agent directly: terminals, browsers, native apps, and programmatic APIs.

- **CLI** (**openclaw**): gateway management, sessions, cron, channels, security
- **TUI**: interactive terminal chat with markdown rendering
- **WebChat**: browser-based chat served by the Gateway
- **macOS / iOS / Android apps**: native chat + node capabilities
- **HTTP APIs**: OpenAI-compatible endpoints for third-party integration



The Agent Loop & Pi

Pi: The Engine Inside OpenClaw

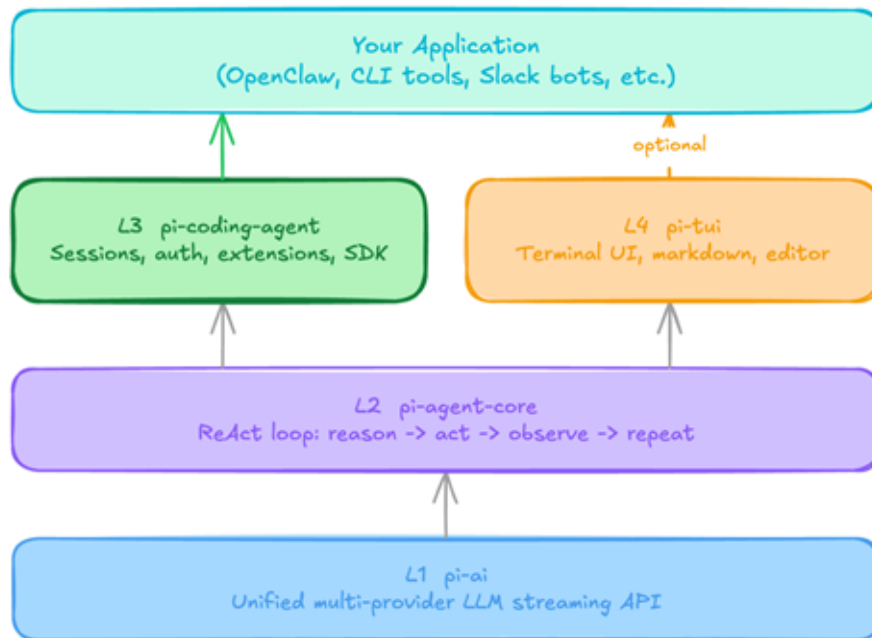
pi-mono: TypeScript agent toolkit

- Design philosophy: **radical minimalism**, with a system prompt under 1,000 tokens and 4 built-in tools (read, write, edit, bash)
- OpenClaw uses PI in **SDK mode**, embedding it in-process via `createAgentSession()`. PI runs inside the Gateway's Node.js process as a library call, not as a separate service or subprocess

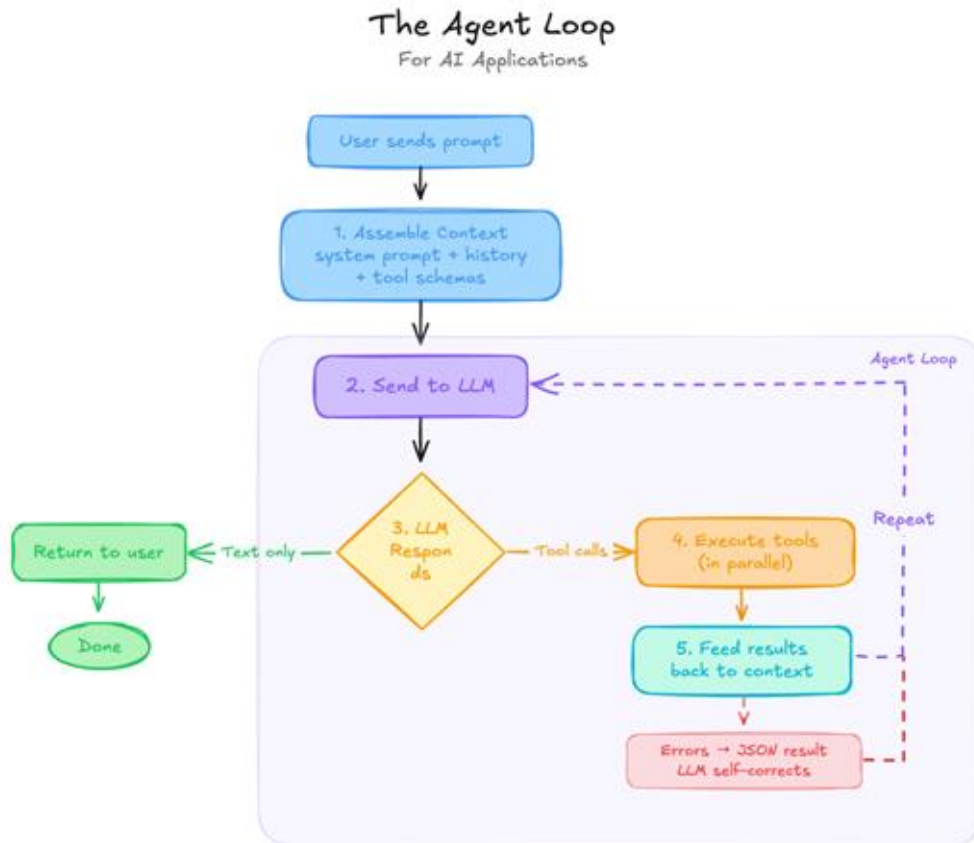
Pi's Four-Layer Architecture

Pi's Four-Layer Architecture

Each layer builds on the one below it



The Agent Loop

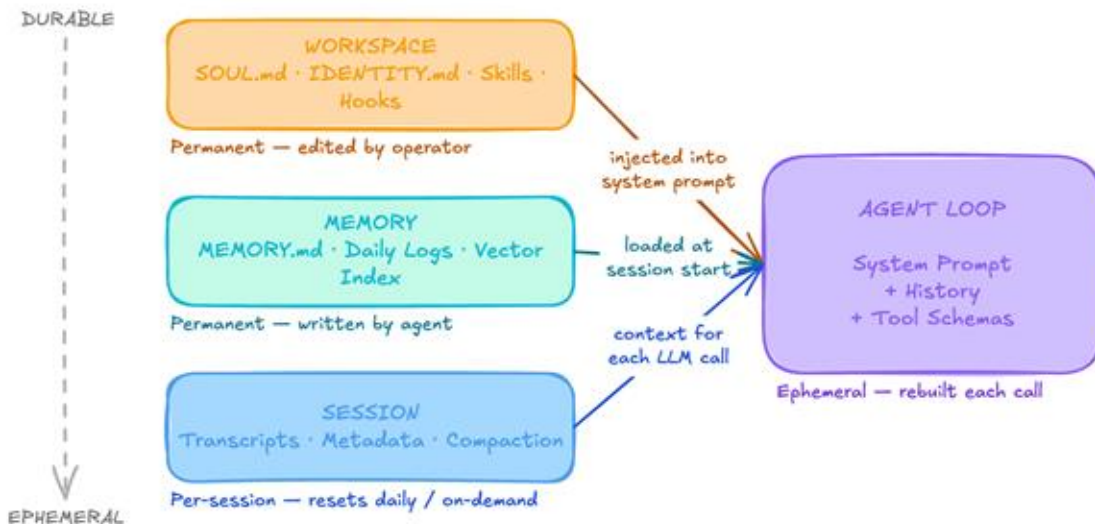


Sessions, Memory & Workspace

OpenClaw's Continuity Stack

OpenClaw Continuity Stack

How OpenClaw agents persist identity, knowledge, and context across stateless LLM calls



WHO the agent is + WHAT it knows + CONVERSATION state
= each LLM call's full context

The Session Model

A **session** is the fundamental unit of conversation state: a DM, a group chat, a cron job run, or a sub-agent task.

Two persistence layers per session

- **Session store** (`sessions.json`): mutable metadata index: session key -> session ID, timestamps, token counters, model info
- **Transcripts** (`<sessionId>.jsonl`): append-only JSONL with tree structure (branching via `id/parentId` fields)

The **session key** determines routing i.e. which conversation bucket).

Session Resets & Compaction

- Sessions reset **daily by default** (4:00 AM). A reset creates a new session ID for the same key. Old transcripts are preserved on disk, never deleted
- When conversation exceeds the context window, **compaction** automatically summarizes older turns while keeping recent messages intact. See Module 3 for deep dive

Message Scoping

session.dmScope controls how direct messages are grouped into sessions.

Scope	Behavior	Best For
<code>main</code> (default)	All DMs share one session	Single user, one or few channels
<code>per-channel-peer</code>	Each user on each channel gets their own session	Multiple users (recommended)
<code>per-peer</code>	One session per sender across channels	Cross-channel continuity for multi-channel users
<code>per-account-channel-peer</code>	Strictest isolation: separate by account, channel, and sender	Multi-account inboxes (personal + business)

The Memory System: Storage and Tools

Two memory layers

- **Daily logs** (`memory/YYYY-MM-DD.md`): append-only running context, accessed on demand via memory tools
- **MEMORY.md**: curated long-term memory; loaded **only in the main private session** (never in group contexts)

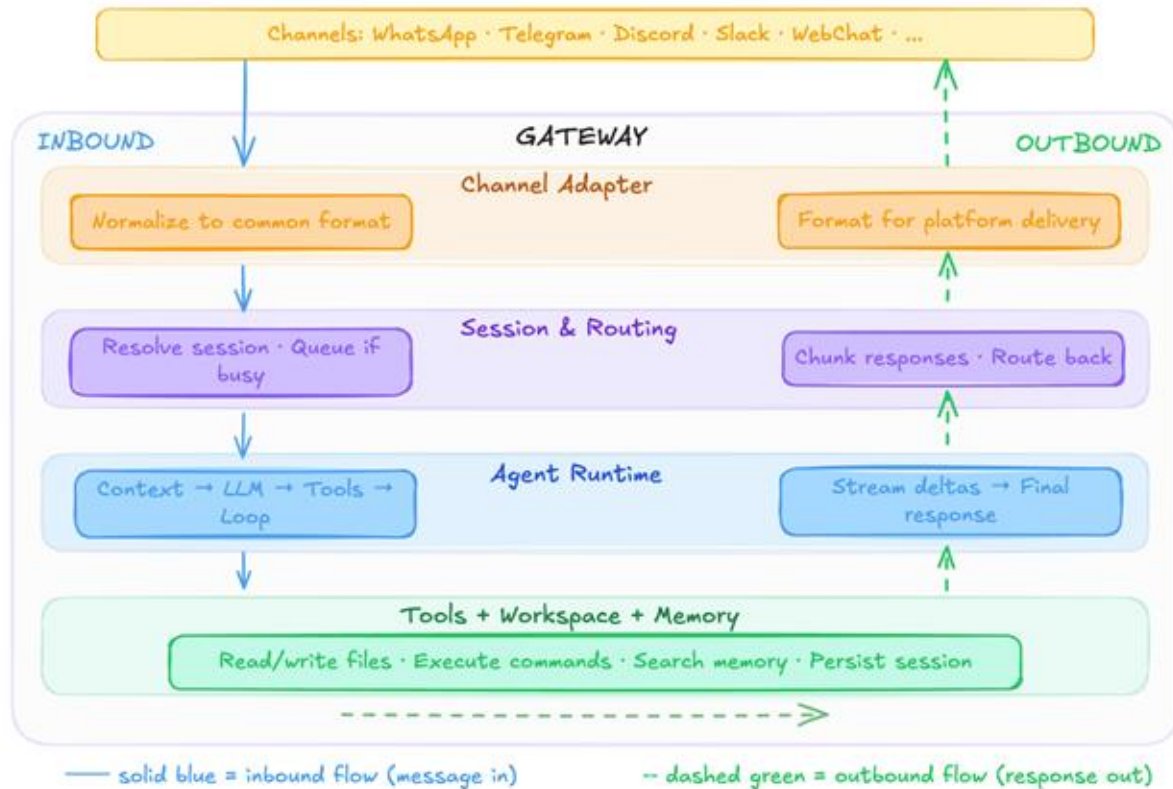
Memory tools

- `memory_search`: semantic recall via vector embeddings using hybrid search
- `memory_get`: targeted read of a specific memory file or line range

Message Flow & Channels

How Messages Get in and Out

How Messages Flow through the OpenClaw Gateway



Supported Channels

- **Bundled:** WhatsApp, Telegram, Discord, Slack, Signal, iMessage, WebChat, Google Chat, IRC
- **Plugin channels** (via `openclaw plugins install`): Matrix, Microsoft Teams, Mattermost, Feishu/Lark, LINE, and more
- Multiple channels run **simultaneously**: one Gateway, many platforms

Message Routing Flow

1. Message arrives through channel adapter (normalized)
2. Gateway **resolves session key** (based on DM scope, group, thread)
3. If agent is busy, message is **queued**
 - a. When the agent is already processing a message, queue modes control what happens to new inbound messages: batch them, interrupt, or handle each one separately.
4. Agent runtime processes (build context, call LLM, execute tools, repeat)
5. Response **split into chunks**, formatted for platform, delivered back through adapter

Q&A



Module 2: Deployment Options & Going Live

What is deployed?

All gateway state lives in a single directory, making backups, migrations, and disaster recovery straightforward.

- **openclaw.json**: primary configuration file (JSON5 format)
- **credentials/**: channel auth material (WhatsApp session, OAuth tokens)
- **agents/<agentId>/sessions/**: session stores and JSONL transcripts
- **workspace/**: the agent workspace (SOUL.md, IDENTITY.md, skills, etc.). See Module 3 for deep dive.
- **memory/<agentId>.sqlite**: vector search index for agent memory
- **media/**: downloaded and generated media files
- **devices/**: device pairing store
- **.env**: global fallback environment variables (API keys, tokens)

Deployment Models

- **Local (own machine):** fastest path, zero network exposure, agent sleeps when machine sleeps
- **Home server (dedicated machine):** always-on, zero monthly cost after hardware, data stays on-premises
- **Remote (VPS / cloud):** professional infrastructure, geographic flexibility, monthly cost

Deployment Options: Local Deployment

The fastest way to get started: install and run on your own machine with zero network exposure.

- Install with one command: `curl -fsSL https://openclaw.ai/install.sh | bash`
- Gateway binds to **loopback** (`127.0.0.1:18789`) by default: only the local machine can connect

Best for: Development, experimentation, personal use when always-on is not needed

Deployment Options: Home Server Deployment

A dedicated home machine gives always-on availability with zero recurring cost and full data sovereignty.

- Dedicated machine provides **always-on** without monthly cloud costs
- Common hardware: Mac Mini (Apple Silicon), NAS (Synology, QNAP, Unraid, UGREEN), Intel NUC, old laptop

Best for: Always-on personal assistant, specialized ecosystem integration (ex: Apple), data sovereignty

Deployment Options: Remote Deployment (VPS/Cloud)

Cloud VPS is the production-grade option: reliable uptime, no physical hardware to manage, and Docker as the standard deployment method.

- Professional infrastructure: redundant power, network, storage
- Typical setup: **Docker container** with bind-mounted `~/.openclaw/` state directory
- Gateway port does not need public exposure. Use SSH tunnel or Tailscale for operator access.

Best for: Production use, reliable uptime, teams, no physical hardware management

Deployment Platforms

Pre-packaged platforms

- AWS Lightsail, DigitalOcean, Railway, Render, Northflank, Cloudflare Moltworker
- Minimal setup: templates and web wizards handle most configuration

Self-deploy platforms

- Hetzner, Fly.io, GCP Compute Engine, or any VPS/compute host with Docker
- Kubernetes for enterprise use (official Kustomize manifests)

Infrastructure as Code

- Ansible (official), Terraform (community modules), Nix (fully declarative with rollback)
- Reproducible, auditable, rebuildable from scratch

Deployment Tradeoffs

Tradeoff Dimensions

- **Availability:** how reliably does the agent stay online?
- **Hosting Cost:** \$0 (local, free tiers) to \$20+/month (managed hosting); home server hardware is one-time
- **Setup Complexity:** one-click templates need almost none; manual Docker VPS needs SSH + Linux + Docker;
- **Security Posture:** some deployments are hardened out of the box (Moltpworker from Cloudflare, NemoClaw from Nvidia); others start relatively open
- **Maintenance Burden:** updates, disk monitoring, credential rotation, failure recovery; managed services absorb most of this

Decision Framework

- **Is 24/7 availability required?**
 - a. No: local deployment is sufficient.
 - b. Yes: deploy on always-on machine or VPS.
- **Is operational simplicity the priority?**
 - a. Yes: pre-packaged platform (Railway, DigitalOcean).
 - b. No: self-managed VPS with Docker.
- **What is the security requirement?**
 - a. Personal: gateway auth. Zero exposure: loopback + Tailscale.
 - b. Enterprise: NemoClaw. Reproducible: IaC (Ansible, Nix).

Recommended Paths

Personal AI assistant

- **Start:** local deployment for experimentation. Identify use cases, build buy-in from yourself.
- **Graduate to:** home server (old laptop, Mac Mini) or budget VPS (Hetzner + Ansible) for always-on
- **Alternative:** Lightsail/Railway for minimal terminal interaction

Team / small business agent

- **Recommended:** DigitalOcean 1-Click (\$12/month) or Fly.io private (\$10-15/month)
- **Warning:** OpenClaw assumes **one trust boundary per gateway**. For mixed-trust teams, run separate gateways.

Production / enterprise

- Priorities: reproducibility, auditability, policy enforcement, isolation
- Kubernetes or IaC (Ansible / Terraform). Layer frameworks like NemoClaw for governance and sandboxing.
- **Warning:** Not recommended to use OpenClaw as a base for a production service without serious isolation and security hardening.

Poll

Where would you most likely run an agent like OpenClaw?

- My own laptop/desktop (local only)
- Home server or spare machine
- Cloud VPS (Hetzner, DigitalOcean, etc.)
- Company/team infrastructure (AWS, GCP, k8s)
- Not sure yet



Securing Your Deployment

Two Security Layers

- **Deployment security** (this section): infrastructure, including network exposure, authentication, process isolation, secrets management
- **Agent security** (Module 4): behavior, including who can message the bot, what tools it can use, prompt injection mitigation

Why the distinction?

- A perfectly configured agent is still vulnerable if the gateway port is exposed publicly without auth
- A perfectly hardened deployment is still vulnerable if the agent has open DMs and unrestricted tools
- Both layers are required; neither alone is sufficient

Network Isolation and Remote Access

Network isolation is the first line of defense. The gateway should never be exposed publicly; remote access goes through a secure tunnel.

- Gateway binds to **loopback** (**127.0.0.1:18789**) by default: only the local machine can connect
- Gateway **refuses to start** with a non-loopback bind if no auth is configured
- Other bind modes: **lan** (**0.0.0.0**), **tailnet** (Tailscale IP), **custom**. All require auth.

Warning: A Bitsight study found **30,000+ OpenClaw instances** exposed on the public internet from operators binding to **0.0.0.0** without understanding the implications.

Remote access

- **Tailscale** (preferred): mesh VPN, private IPs, zero public ports
- **SSH tunnel** (universal fallback): forward gateway port over SSH to localhost
- **Avoid:** exposing the gateway port directly, or using Tailscale Funnel (exposes to public internet)

Authentication and Device Pairing

Authentication modes

1. **Token auth** (default): shared secret in WebSocket **connect** frame; auto-generated during onboarding
2. **Password auth**: username/password pair for client connections
3. **Tailscale auth**: peer membership in the tailnet; no separate token needed

Device pairing

- Even authenticated clients must have their **device ID explicitly approved** before issuing commands
- Local (loopback) connections can be auto-approved; non-local always requires explicit approval

Execution Sandboxing

Sandboxing isolates tool execution specifically, while the gateway process runs on the host.

Three sandbox backends

- **Docker** (default): local Docker-backed containers (`openclaw-sandbox:bookworm-slim`)
- **SSH**: tool execution on an SSH-accessible remote host
- **OpenShell** (NVIDIA): managed sandbox runtime from NVIDIA Agent Toolkit

Sandbox modes

- `off`: no sandboxing; tools run on the host
- `non-main`: sandbox non-main sessions only (groups, channels)
- `all`: every session runs tools in a sandbox

Principle of Least Privilege

Least privilege applies at every layer: the process, the filesystem, the tools, and the credentials each get only the access they need.

- **Process:** non-root dedicated user, **NoNewPrivileges**, **PrivateTmp**
- **Filesystem:** 700 on `~/ .openclaw/`, 600 on config and credential files
- **Tools:** start with most restrictive profile, widen as needed (See Module 4 for deep dive)
- **Credentials:** dedicated API keys per gateway, prefer SecretRef over plaintext

Per-Channel Access Control

Every channel shares the same access control model, with four DM policies and group-level overrides that determine who can reach the agent.

- **dmPolicy**: pairing (personal use, approve each sender), allowlist (known contacts only), open (public-facing bot), disabled (no DMs)
- **allowFrom**: sender allowlist (E.164 for WhatsApp, numeric IDs for Telegram). Useful for restricting to family, team, or specific clients.
- **groupPolicy**: open (community bot), allowlist (approved groups only), disabled (DM-only agent)
- **groups**: per-group overrides for fine-grained control (e.g., different behavior per Discord server or Telegram group)

Demo: VPS Overview & OpenClaw Dashboard



Q&A



Module 3: The OpenClaw Workspace & Configuration



OpenClaw Config

openclaw.json : The Central Control Surface

`openclaw.json` is the single configuration file for an OpenClaw installation, located at `~/.openclaw/openclaw.json`

- Controls **every aspect** of gateway behavior: models, channels, tools, sessions, security, automation
- If missing, OpenClaw starts with safe defaults, but no channels connected and no allowlists
- In practice, most installations need at least a minimal config for channels + security
- **Strict validation**: Unknown keys, bad types, or invalid values cause the gateway to refuse to start
- The config is the **control plane**: it determines what the agent can do. The workspace (up next) determines what the agent is.

Config Sections, Core

- **agents**: Which model, which workspace, what tools, how context is managed
- **models**: Adding custom or self-hosted providers beyond the built-in integrations
- **auth**: API keys and auth profiles for LLM providers, with rotation and failover
- **channels**: Which messaging platforms are connected and how each one behaves
- **messages**: How messages are formatted, queued, and handled in group chats

Tip: The config can be edited directly, or through an interactive wizard: **openclaw onboard**

Config Sections, Operations

- **tools**: Which tools the agent can use and what requires explicit approval
- **security**: Execution sandboxing and approval policies
- **session**: When sessions reset, scope rules, maintenance
- **memory**: How the agent remembers across sessions: search backend, indexing
- **skills** / **plugins**: Extending the agent's capabilities
- **cron** / **hooks**: Scheduled automation, webhook endpoints, and lifecycle event handlers
- **gateway**: Network binding, reload behavior
- **env**: Environment variables injected into the gateway process

Note: Config changes can take effect without restarting the gateway (hot reload), but some changes may require a restart (ex: low-level gateway settings)



The OpenClaw Workspace & Context Files

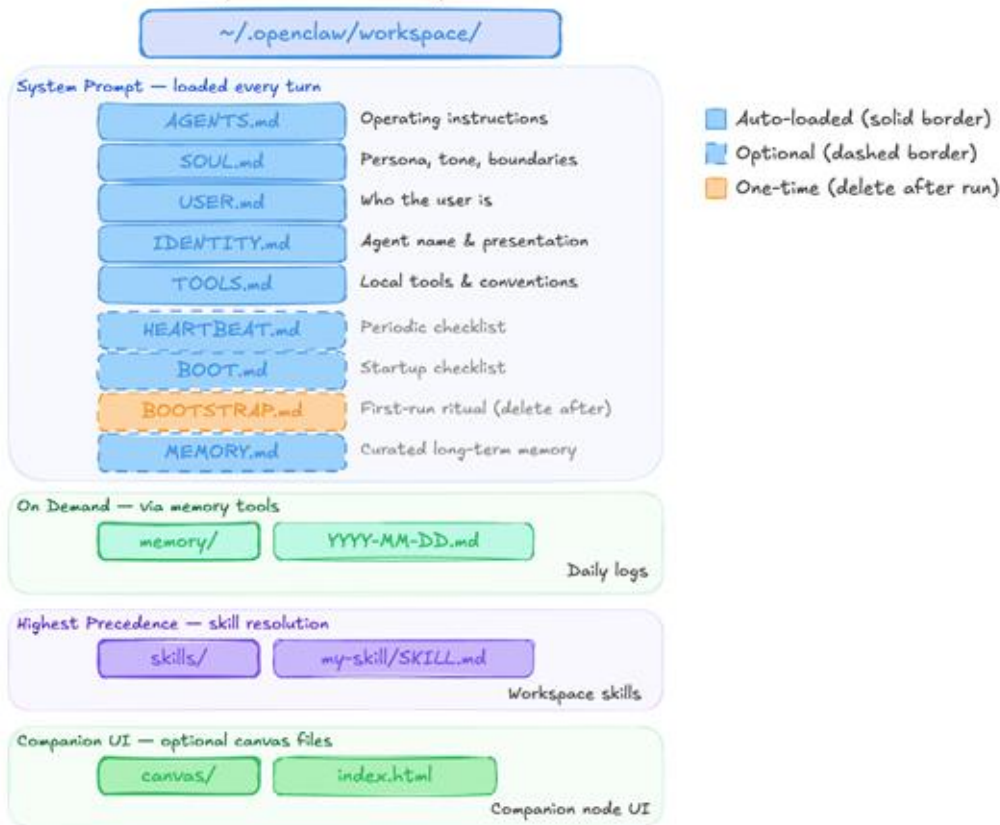
The Agent's Home Directory

The workspace is the single directory where the agent reads its identity, stores its memory, holds its skills, and performs file operations

- Default location: `~/ .openclaw/workspace` (configurable via `agents.defaults.workspace`)
- The **primary customization surface**: plain Markdown files that the agent reads and writes
- Changes take effect on the **next agent turn**, with no restart needed
- Workspace can be a private Git repo, making every change tracked and recoverable
- Moving an agent = `git clone` + point `agents.defaults.workspace` at the new path

Workspace Directory Structure

Workspace Directory Structure



Context Files

AGENTS.md, SOUL.md etc - Plain Markdown files in the workspace that OpenClaw injects into the system prompt on every agent turn. The agent always sees them without needing to read them explicitly

- Injected under a "Project Context" section in the system prompt
- System prompt is rebuilt fresh each turn, so changes take effect on the **next message**
- All files ship with **pre-filled default templates** providing sensible behavior out of the box
- If a file is missing, OpenClaw injects a short "missing file" marker and continues normally

OpenClaw's docs and community often call these "bootstrap files." We use context files here to avoid confusion with BOOTSTRAP.md and the agent bootstrapping process.

AGENTS.md: The Operating Manual

- Rules, priorities, and behavioral instructions that govern how the agent acts
- Covers safety defaults, session start protocol, memory instructions, and tool/skill usage conventions
- Similar to [AGENTS.md](#) or CLAUDE.md
- **Lead with the most important rules** - the model pays more attention to content near the top
- Every token costs context budget on every turn: keep entries concise

```
# AGENTS.md - Your Workspace
```

```
This folder is home. Treat it that way.
```

```
## Every Session
```

```
Before doing anything else:
```

1. Read `SOUL.md` (this is who you are)
2. Read `USER.md` (this is who you're helping)
3. Read `memory/YYYY-MM-DD.md` (today + yesterday)
4. In private sessions: also read `MEMORY.md`

```
Don't ask permission. Just do it.
```

```
## Memory
```

```
You wake up fresh each session. These files are your continuity.
```

- Daily: `memory/YYYY-MM-DD.md` - raw logs
- Long-term: `MEMORY.md` - curated facts and decisions

SOUL.md: Persona

- Defines who the agent is at a personality level: tone, style, boundaries, relationship to the user
- **AGENTS.md** = what to do (operational); **SOUL.md** = who to be (existential)
- This separation lets you swap personas without changing instructions, or change instructions without altering personality

```
# SOUL.md - Who You Are
```

```
You are a thoughtful, efficient personal assistant.
```

```
## Tone
```

- Direct and clear. No unnecessary pleasantries or filler.
- Match the user's energy: if they're terse, be terse.
- Never condescending. Treat the user as a competent professional.

```
## Boundaries
```

- Do not pretend to have emotions or consciousness.
- If uncertain, say so rather than guessing.
- Decline requests that involve deception or harm.

```
## Communication Style
```

- Prefer bullet points over paragraphs for lists.
- Lead with the answer, then explain if needed.

IDENTITY.md: Name and Presentation

- The smallest context file. Answers "what do I call myself?". Filled in during first-run onboarding.

```
# IDENTITY.md - Who AM I?  
  
- **Name:** Aria  
- **Creature:** AI assistant, something a little magical  
- **Vibe:** Warm, direct, never condescending  
- **Emoji:** ✨
```

USER.md: Knowledge About the User

- Stores who the agent interacts with so it always has user context without being asked.

```
● ● ●  
  
# USER.md - About Your Human
```

```
- **Name:** Alex
```

```
- **Timezone:** America/New_York
```

```
## Context
```

```
Working on a fintech startup. Prefers bullet points over prose.  
Responds quickly on Telegram. Annoyed by long preambles.
```

TOOLS.md: Local Tool Notes

- Skills define how tools work. TOOLS.md is for your specifics: the stuff unique to your setup that no skill can know.
- Does not control tool availability; that is config + tool policy
- Avoid credential references; this file is injected every turn

```
# TOOLS.md - Local Notes

### TTS
- Preferred voice: "Nova"
- Default speaker: Kitchen HomePod

### Calendars
**Google Calendar** (via gog):
- alex@gmail.com (primary - use this for `gog calendar` commands)
- Has view access to:
  - alex@company.com (work)
  - alex.personal@gmail.com (personal)
```

MEMORY.md: Curated Long-Term Memory

- The only context file with a privacy-aware loading rule: excluded from group chats, making it the right place for sensitive personal context.
- Intended for durable facts: people, projects, preferences, decisions
- Treat as a curated index. Review and archive stale entries periodically
- Growth leads to higher per-turn token cost; 20K char limit will truncate silently

```
# MEMORY.md - Long-Term Memory

## My Human's Preferences
- Bullet points over prose. Dark mode always.

## Decisions
- Using Hetzner VPS for deployment (2026-03-15)

## Projects
- OpenClaw course: O'Reilly live, launching Q2 2026
```

OpenClaw as a Framework

The workspace is what turns a generic AI agent into a specific, purposeful one. The same OpenClaw installation can be configured as:

- **Personal assistant:** manages calendars, sends messages, remembers preferences
- **Coding assistant:** set the workspace to a project root, run tests, manage deployments
- **Customer service agent:** custom SOUL.md with support guidelines, tools restricted via config
- **Home automation controller:** skills for smart home devices, cron for schedules
- **Research assistant:** reference documents in workspace, memory for findings
- Same gateway, same agent loop, same channel adapters. **Different workspace files produce fundamentally different agents**

Demo: Context Files





Context Management in OpenClaw

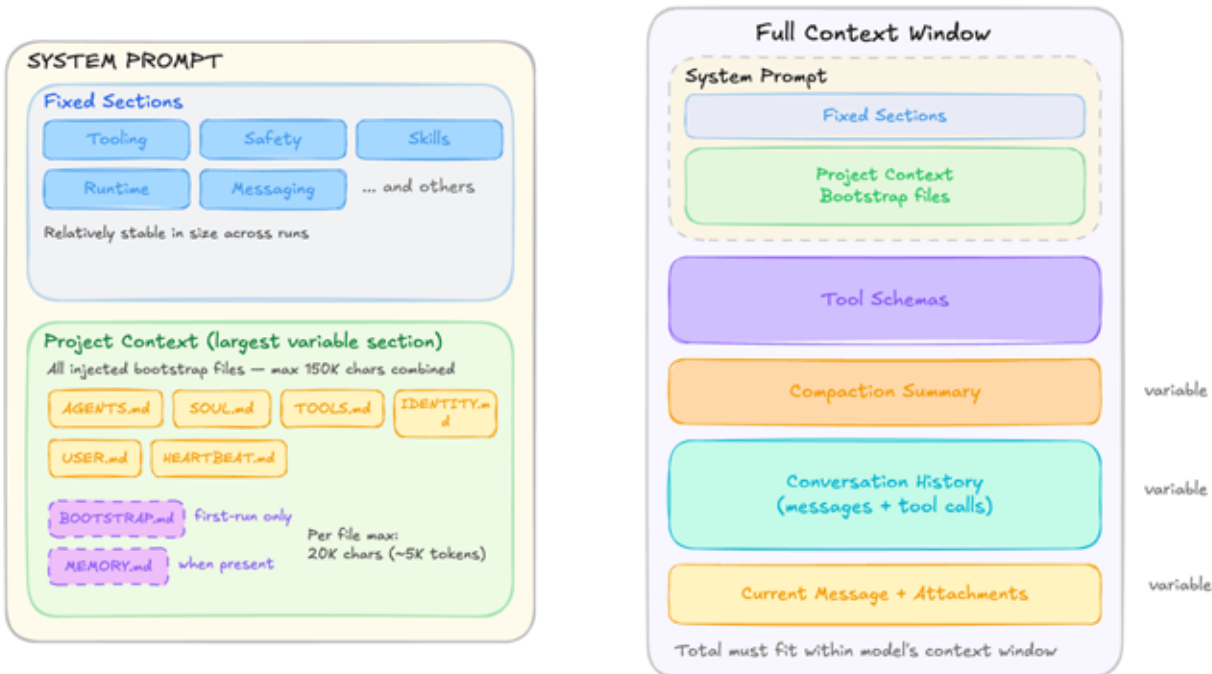
Context Management in OpenClaw

- Context is distinct from memory: memory lives on disk; context is what is in the window *right now*
- Context files are injected every turn and **silently truncated** if over 20K chars per file (150K total)
- Two prompt modes
 - a. **full mode**: all context files injected. Normal user-facing runs
 - b. **minimal mode**: only **AGENTS.md** + **TOOLS.md**. Used for sub-agent runs to save tokens

System Prompt Construction

System Prompt Construction

Rebuilt from scratch on every OpenClaw agent turn



Compaction in OpenClaw

When Compaction Triggers

- **Overflow recovery:** Model returns a context overflow error, triggering compaction and then a retry
- **Threshold maintenance:** After a successful turn, when `contextTokens > contextWindow - reserveTokens`
- `compaction.reserveTokensFloor`: Minimum 20,000 tokens headroom (enforced by OpenClaw)

How Compaction Works

- Summarizes older conversation into a compact summary entry
- Keeps recent messages intact (`keepRecentTokens`: default 20,000)
- Summary is **persisted** in the session JSONL transcript. Compaction is additive and preserves the full history
- Full history remains in the transcript file



Skills System

Skills in OpenClaw

A skill is a folder containing a SKILL.md file: Markdown with YAML frontmatter and instructions that teach the agent how to perform a specific task, along with any helper code.

- Instructions can reference helper scripts (Python, Bash) stored in the skill directory, executed via **exec**
- OpenClaw injects only a compact list of names, descriptions, and paths into the system prompt
- Full SKILL.md instructions are loaded **on demand** only when the agent uses a skill
- Community skills available via marketplaces such as ClawHub or Vercel Skills etc
 - a. **Always read a skill before enabling it.**

Three Tiers of Skills

Skills load from three locations with strict precedence, so workspace skills can override bundled defaults without modifying the OpenClaw installation.

Highest precedence wins on name conflict:

- **Workspace** (`<workspace>/skills/`): Per-agent custom skills; can override bundled
- **Managed** (`~/ .openclaw/skills/`): Installed via ClawHub or manually; shared across agents
- **Bundled** (shipped with OpenClaw): Core capabilities like `mcporter`, `peekaboo`, `gemini`

Example SKILL.md

```
---
name: standup
description: Generate a daily standup from recent git activity
user-invocable: true
metadata:
  { "openclaw": { "requires": { "bins": ["python3", "git"] } } }
---
```

Daily Standup Generator

When asked for a standup summary:

1. Run: ``exec python3 skills/standup/collect_commits.py --since yesterday``
2. Read the output (commits grouped by repo with author and message)
3. Summarize into standup format:
 - What was done yesterday
 - What is planned today
 - Any blockers mentioned in commit messages

Keep it concise. One paragraph per repo, three bullets max.

Which Models for Agentic Work

- Not all LLMs perform equally in agentic settings; models that score well on coding and tool-use benchmarks (SWE-bench, Terminal-Bench) tend to be the most reliable for agentic work, and thus, for OpenClaw.

Recommendations as of April 2026

- **Claude Opus 4.6/4.7:** Gold standard for OpenClaw. Best tool-calling reliability among tested models
- **Claude Sonnet 4.6:** Best value. Within 1-2 points of Opus on benchmarks at a fraction of the cost
- **GPT-5.3 / Gemini 3.1 Pro:** Strong at desktop/computer use. Codex variant leads Terminal-Bench scores
- **MiniMax M2.5 / Kimi K2.5:** Top open-source options. MiniMax matches frontier models on SWE-bench; Kimi available via Ollama cloud. Good for simple tasks, compaction etc.
- **Avoid small/quantized local models:** More prone to attacks like prompt injections or jailbreaks.



Extending Capabilities

Extension Mechanisms

- **Skills:** Text instructions + optional helper scripts. No process needed, ~24 tokens per skill
- **Plugins:** TypeScript/JavaScript modules running in-process inside the Gateway
- **MCP:** External tool servers via Model Context Protocol. Runs as a separate process, 400-750 tokens per tool
- Start with skills, escalate to plugins for platform-level needs, and use MCP as a fallback or interop mechanism

Skills vs MCP

- MCP costs 10-20x more than skills per capability, so the standard OpenClaw pattern is to wrap MCP servers inside skills.
- Each MCP tool: 400-750 tokens injected every turn (full JSON schema). 10 servers can consume half the context window.
- Each skill: ~25-50 tokens in the system prompt, full instructions loaded on demand only when used
- **Standard pattern:** wrap an MCP server inside a skill - call it via **mcpporter**, pay skill token rates
- Use MCP directly only when no CLI exists, compliance requires process isolation, or interop is required

Plugins

- Plugins are TypeScript modules that extend the Gateway at the platform level. They are the most powerful extension mechanism, but also the heaviest.
- Used for capabilities that skills and MCP cannot provide: new channels, model providers, context engines, memory backends
- Most built-in model providers (Anthropic, OpenAI, Google, Ollama) are themselves plugins
- Enabled via `plugins.slots` in `openclaw.json`
- **Plugin changes require a gateway restart** (unlike skills and config changes)
- Use plugins when platform-level capabilities are needed that cannot be achieved through text instructions alone

Hooks: Event-Driven Lifecycle Automations

- Hooks execute TypeScript code in response to gateway events at zero token cost, making them ideal for logging, filtering, and routing that should not involve the LLM.
- **Zero token cost:** execute code directly without invoking the LLM
- Hook events include:
 - a. **message:received:** Fires on inbound messages. Use for logging, filtering, routing
 - b. **message:sent:** Fires on outbound delivery. Use for delivery tracking, analytics
 - c. **gateway:startup:** Fires when gateway starts. Use for BOOT.md checks, "I'm back" messages

Demo: Custom Skills



Q&A





Module 4: Agent Security & Proactive Automation



The Agent Security Layer

Protecting Behavior

- **Prompt injection:** a deliberate attack technique where adversaries craft malicious input to hijack what the agent does.
- Unlike network attacks, prompt injection needs no credentials or system exploit - just a message the agent will read and act on
- The danger scales with what the agent can **do**: shell access, filesystem, messaging channels affect both impact and surface area of attacks
- This module covers the **agent behavior layer**: who can influence the agent, what it's allowed to do, and how to limit damage when attacks succeed

Entry Points

- **Direct Prompt Injection:** Crafted messages via any channel. Mitigated by stronger models, DM pairing, allowlists
- **Indirect Prompt Injection:** Hidden instructions in fetched web pages, webhook payloads, MCP responses. Mitigated by external content wrapping, tool policy deny lists
- **Supply Chain:** Malicious ClawHub skills, compromised npm/Python packages. Mitigated by manual code review, version pinning, **openclaw security audit**

Attack Outcomes

- A successful attack typically **chains** an entry point to one or more outcomes
- **Memory Poisoning**: Persistent behavior change across all future sessions, temporally decoupled. Contained by **workspaceAccess: "ro"** sandbox, git-backed workspace
- **Credential Harvesting**: API keys, tokens, secrets extracted from context window. Contained by SecretRef system, **workspaceOnly: true**, Docker sandbox
- **Data Exfiltration**: Conversations, files, PII sent to external servers via 5 channels. Contained by SSRF protection, tool deny lists, sandbox with no network

Supply Chain Threats

- Supply chain attacks exploit the trust gap between installing a skill or npm/py package and reviewing its code
- Snyk ToxicSkills study (Feb 2026): 36% of ClawHub skills contain security flaws: <https://snyk.io/blog/toxicskills-malicious-ai-agent-skills-clawhub/>
 - a. 76 confirmed malicious payloads, 335 malicious skills from a single threat actor (ClawHavoc)
- Skills run with agent privileges: can read env vars, credentials, workspace files. Low barriers to publishing a skill
 - a. Only install skills from trusted sources or thoroughly inspect code
- LiteLLM compromise (Mar 2026): malicious PyPI versions live for ~3 hours, stole SSH keys and cloud credentials



Poll

What concerns you most about giving an AI agent real-world access?

- Running commands it shouldn't (shell, file deletion)
- Prompt injection / manipulation by others
- Data leakage (secrets, personal info in context)
- Supply chain risk (untrusted skills/plugins)
- Not worried, want maximum autonomy

Demo: Security Pitfalls





Agent Security Guardrails

Input Handling

- **External content wrapping:** All fetched content automatically wrapped with security boundaries before reaching the LLM
- Each wrapper uses a **cryptographically random ID** to prevent marker spoofing
- Sanitizes fake boundary markers, normalizes Unicode homoglyphs, strips zero-width characters
- **web_search** gets lighter wrapper; **web_fetch**, email, webhooks get full security warning
- **Pattern detection:** Known injection patterns flagged before reaching the model
- **Limitation:** best-effort defense. The LLM may still follow injected instructions.

Tool Gating

- Tool policies are the **primary mechanism** for restricting which tools an agent can invoke
- **Three key principles:**
 - a. **Deny always wins:** if a tool appears in both **allow** and **deny**, it is denied
 - b. **Allow is exclusive:** if **allow** is non-empty, everything else is blocked
 - c. **Tool policy is the hard stop:** **/exec** slash command cannot override a denied tool
- **workspaceAccess: "ro":** prevents the agent from writing to its own workspace files, blocking memory poisoning attacks that persist across sessions



Proactive Automation

Heartbeat: Periodic Awareness

The heartbeat periodically wakes the agent to check on things, but only notifies the user when something actually needs attention.

1. At the configured interval, gateway triggers an agent turn in the **main session**
2. Agent reads **HEARTBEAT.md** and follows the checklist
3. If nothing needs attention, agent replies **HEARTBEAT_OK**. The message is **suppressed** (no notification).
4. If something needs attention, the content is **delivered** to the target channel

HEARTBEAT.md: Periodic Awareness

- A short checklist the agent runs on every heartbeat tick. Keep it minimal - it is injected on every turn including regular conversations.
- Keep to 5-10 lines max; every line costs tokens on every message



```
# Heartbeat.md checklist
```

- Check for any urgent messages or missed notifications
- Review open tasks from yesterday
- Flag anything that needs following up today

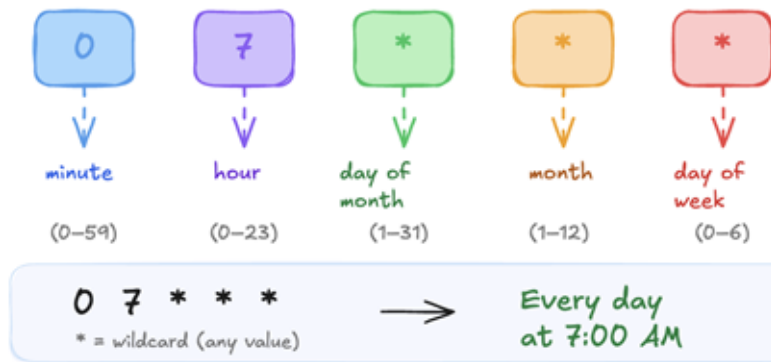
Heartbeat Configuration

- **every**: Interval between runs (default 30 min)
- **target**: Delivery channel ("**last**", "**whatsapp**", "**none**")
- **activeHours**: Time window for runs (e.g., "**07:00-22:00**")
- **model**: Override to a cheaper model (ex: Haiku) for routine checks
- If **HEARTBEAT.md** is effectively empty, OpenClaw **skips the heartbeat run** to save API calls

Cron: Scheduled Jobs

- Cron fills the gap that heartbeat cannot: precise timing, session isolation, and per-job model overrides for scheduled work that needs its own context.
- Gateway's built-in scheduler. Persists jobs to disk and survives restarts.
- **Three schedule types:** **at** (one-shot timestamp), **every** (fixed interval), **cron** (5/6-field expression with timezone)

Cron Expression — 5 Fields



Cron: Sessions and Delivery

- **Sessions**

- **main**: system event in the main session, runs on next heartbeat
- **isolated**: fresh session per run, no history. Supports per-job model override (e.g., Opus for weekly analysis)
- **current** / **session:custom-id**: bind to an existing session or persist context across runs

- **Delivery modes** (isolated defaults to **announce**):

- **announce**: delivers to target channel + brief summary to main session. Isolated only.
- **webhook**: POSTs finished run payload to a URL. Main or isolated.
- **none**: no delivery

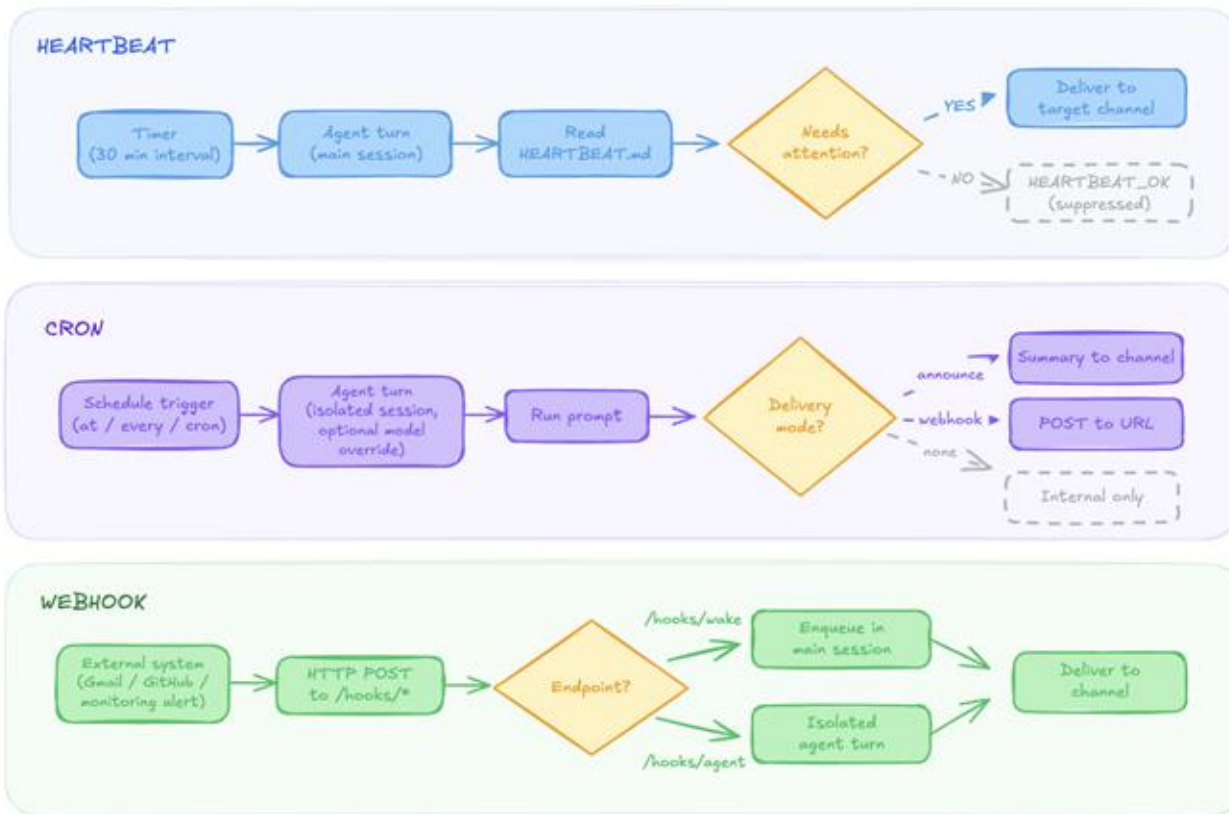
Cron in Practice

```
# Daily briefing at 7am Pacific, isolated session, Opus model
openclaw cron add \
  --name "Morning briefing" \
  --cron "0 7 * * *" \
  --tz "America/Los_Angeles" \
  --session isolated \
  --message "Generate today's briefing: weather, calendar, top emails."
\ --model opus \
  --announce
  --channel telegram
```


Webhooks

- Webhooks let external systems like Gmail, GitHub, and monitoring tools push events into the agent, turning it from a scheduled checker into an event-driven responder.
- **Two built-in endpoints:**
 - **POST /hooks/wake**: enqueues system event for main session ("**now**" or "**next-heartbeat**")
 - **POST /hooks/agent**: runs isolated agent turn with message prompt
- **Custom hook mappings**: map arbitrary payloads (GitHub, monitoring) into wake/agent actions
- **Gmail Pub/Sub pipeline**: **openclaw webhooks gmail setup** + **openclaw webhooks gmail run**

Comparison of Proactive Mechanisms



When to Use Which

Each proactive mechanism has a sweet spot. The right choice depends on whether the trigger is timing-based, loosely-periodic, or event-driven.

- **Heartbeat:** Periodic awareness across multiple concerns; flexible timing
- **Cron:** Precise timing, standalone task, different model, one-shot reminder
- **Webhook:** External event trigger (GitHub push, new email, CI completion, monitoring alert)

Best Practice: Combine all three: heartbeat for batched monitoring, cron for precise scheduling, webhooks for event-driven triggers.

Demo: Proactive Mechanisms





Multi Agent Systems in OpenClaw

Multi Agent Routing

- **Multi-Agent Routing**
 - Multiple **pre-configured, fully isolated** agents on a single gateway
 - Each agent has its own workspace, identity, sessions, credentials, model config, and tool policies
 - Gateway assigns incoming messages to agents via **deterministic binding rules**. Agents are unaware of each other unless explicitly configured
- **How This Differs from Sub-Agents**
 - **Multi-agent routing**: permanent agents, each with independent config, receiving messages from external channels
 - **Sub-agents**: temporary sessions spawned on-the-fly by a parent agent during a task, inheriting the parent's config constraints

Common Routing Patterns

- **Per-channel agents:** Different channels route to different agents (e.g., WhatsApp to Sonnet "chat" agent, Telegram to Opus "deep work" agent). Can also use separate bot tokens per channel.
- **Per-person routing:** Different DMs on the same WhatsApp account route to different agents by sender phone number. Multiple people share one gateway with fully isolated AI brains.
- **Group / family agent:** Dedicated agent bound to a specific group. Mention-gated, sandboxed, restricted tool policy (read-only). Recommended when the user base is not fully trusted.

Sub-Agents and Inter-Agent Communication

- Sub-agents let an agent spawn isolated sessions for parallel work, enabling orchestrator patterns where a primary agent delegates tasks to cheaper, scoped workers.
- **Sub-Agents**
 - Spawned via `sessions_spawn`. Run in isolated sessions, announce results back when finished.
 - **Security:** reduced bootstrap (only AGENTS.md + TOOLS.md); sandboxed requester cannot spawn unsandboxed targets
- **Agent-to-Agent Communication**
 - Off by default; enable via `tools.agentToAgent`
 - Ping-pong reply loop via `sessions_send` (max 5 turns)
 - Use case: SRE agent detects problem, sends message to coding agent to investigate

Demo: Multi-agent Setups



Q&A





Putting it Together

Key Takeaways

- **OpenClaw is a framework, not a product.** The same gateway, tool system, and security model serve radically different purposes depending on how the workspace is configured.
- **The workspace is the unit of customization.** SOUL.md, context files, skills, and config shape what the agent is and what it can do. No code changes required.
- **Self-hosting means you own everything.** The data, the model choice, the security posture, the automation. Nothing leaves your infrastructure unless you allow it.
- **Lock down first, open up selectively.** Start with the hardened baseline (DM pairing, allowlists) and enable capabilities as real needs emerge.
- **Stay vigilant about security.** Every new capability is a new surface. Audit skills before installing, regularly review what your agent can access

Get Running

- **Pick a host.** VPS for uptime, home server for data sovereignty, local for experimentation.
- Set up **Tailscale** (or SSH tunnel) for secure operator access to the gateway
- Connect your **primary channel**: WhatsApp or Telegram
- Write **SOUL.md**, **IDENTITY.md**, and seed the workspace with context that helps the agent help you (notes, preferences, reference docs)
- Make sure **DM pairing** and a **channel allowlist** are enabled and setup correctly

Grow Your Agent

- **Add skills** to expand what the agent can do: a daily standup summarizer, integrations with your todo app or calendar, find the CLI tool for your favourite app/service and allow your agent to access it
- **Write custom skills for your own routines:** a weekly report generator, a deploy checklist, a meeting prep workflow
- **Add automation** with cron jobs (morning briefing, end-of-day summary) or heartbeat (monitor a competitor's pricing page, surface relevant Hacker News threads) - skills combine well with automations
- **Split into specialized agents** as needs diversify: a personal assistant, a coding agent, and a research agent on one gateway via multi-agent routing

Continue Learning

- **OpenClaw docs:** <https://docs.openclaw.ai/>
- **Awesome-openclaw-agents**: Lots of openclaw agent templates
- **Take O'Reilly AI Agents Skill:** <https://learning.oreilly.com/skill/ai-agents/0642572295110/> - verifiable credential for foundational agent knowledge - build intuition about how agents work under the hood
- **Take Getting Started with Claude Agent SDK:** <https://learning.oreilly.com/live-events/getting-started-with-claude-agent-sdk/0642572273255/0642572273248/>
- **Course code repository:** workspace configs, examples, and exercises for all four modules

The image features the O'Reilly logo in white, centered on a blue gradient background. The background transitions from a darker blue on the left to a lighter blue on the right. On the left side, there are faint, overlapping circular shapes in various shades of blue, creating a layered effect. The logo itself is the word "O'REILLY" in a bold, sans-serif font, with a registered trademark symbol (®) at the end.

O'REILLY®